

SC102 L07：程序化生成——详细学习笔记

涵盖：模型的组成 | 顶点数据 | Mesh 类 | 程序化网格生成 | 地形与柏林噪声 | 草的生成

1 模型是由什么组成的？

[p.4]

3D 模型的核心两组数据是顶点（**Vertices**）和三角形（**Triangles**）。顶点定义空间中的“点”，三角形定义哪些点连接起来形成面。

2 顶点（Vertices）详解

[p.5]

顶点是模型网格的端点。每个顶点包含多种数据：

- **Position（位置）**：Vector3，顶点在物体空间中的坐标
- **UV（纹理坐标）**：Vector2，用于将 2D 纹理映射到 3D 表面上
- **Normal（法线）**：Vector3，顶点朝向（所在面的垂直方向）
- **Color（顶点色）**：可选的顶点颜色
- **Tangent（切线）**：Vector3，用于法线贴图等高级效果

插值（Interpolation）：顶点数据在三角形面内被插值（线性或透视校正）并应用到面上的每一个像素。这就是为什么 4 个顶点色看起来是一个渐变的菱形。

2.1 Position 顶点坐标

[p.6]

每个顶点的位置是一个以物体中心为原点的 Vector3 坐标。这个坐标系是**物体空间（Object/Local Space）**——之后通过 Transform 的平移/旋转/缩放转换到世界空间。

2.2 UV 纹理坐标

[p.7]

UV 是将 2D 纹理映射到 3D 模型上的坐标系统。UV(0,0) 代表纹理左下角，UV(1,1) 代表右上角。U 对应水平方向，V 对应垂直方向。

2.3 Normal 法线

[p.8-9]

法线代表顶点所在面的朝向，是垂直于面的 `Vector3`，指向外侧。

硬边 (Hard Edge) 与软边 (Soft Edge):

- **硬边:** 同一位置有多条法线（每个相邻面各一条）。渲染时产生明显的棱角。立方体的边就是硬边。
- **软边:** 同一位置只有一条法线（所有相邻面的顶点共享一条平均法线）。渲染时表面光滑过渡。球体的顶点就是软边。

2.4 Color 顶点色

[p.10]

每个顶点可以有对应的颜色。颜色只存在于顶点上，面上其他位置通过线性插值获得——所以看起来是渐变效果而非纯色块。

3 三角形 (Triangles)

[p.11]

在计算机图形学中，表达三角形面的方法是：

索引缓冲区 (Index Buffer): 将三个顶点的下标放入一个数组中。如 [1, 4, 3] 表示由第 1、4、3 号顶点组成的三角形。

Unity 中的绕序 (Winding Order): 三角形必须是顺时针的（从正面看）。逆时针的三角形被认为是反面（背面剔除会跳过渲染）。

4 Unity 中的 Mesh 类

[p.13]

`Mesh` 类涵盖了一个模型网格的所有信息，本质上就是一大堆数组：

```
public class Mesh {  
    public Vector3[] vertices;  
    public int[] triangles;
```

```

    public Vector3[] normals;
    public Color[] colors;
    public Vector3[] tangents;
    public Vector2[] uv;
    public Vector2[] uv2;
    // ...
}

```

`MeshFilter` 所做的就是将导入的模型文件转换成一个 `Mesh` 对象。

5 程序化生成网格

5.1 三角形 (Triangle)

[p.14]

最简单的可显示网格。3 个顶点构成 1 个三角形面。

```

Mesh mesh = new Mesh();
Vector3[] vertices = new Vector3[3];
vertices[0] = new Vector3(0, 0, 0); // 左下
vertices[1] = new Vector3(1, 0, 0); // 右下
vertices[2] = new Vector3(0, 1, 0); // 左上
mesh.vertices = vertices;

int[] triangles = new int[3];
triangles[0] = 0;
triangles[1] = 2; // 顺序是 0-2-1 ! (逆时针才是正面的Unity默认规则)
triangles[2] = 1;
mesh.triangles = triangles;

```

为什么不是一一对应？ `vertices` 和 `triangles` 数组存放不同的内容。`vertices` 按索引 0,1,2 存放坐标；`triangles` 按“顺时针绕序”指定哪三个索引构成一个三角形。

5.2 Quad (矩形)

[p.15]

矩形由两个三角形组成，需要 4 个顶点（共享边的顶点可以复用！）和 6 个三角形索引（2 个三角形 × 3 个索引）。

```

vertices[0] = new Vector3(0, 0, 0);
vertices[1] = new Vector3(1, 0, 0);
vertices[2] = new Vector3(0, 1, 0);

```

```
vertices[3] = new Vector3(1, 1, 0);

triangles = new int[] {
    0, 1, 2,    // 三角形1: 左下→右下→左上
    2, 1, 3     // 三角形2: 左上→右下→右上
};
```

为什么需要 6 个 **triangles** 元素？每个三角形需要 3 个索引，2 个三角形 = 6 个。虽然只有 4 个顶点，但两个三角形需要各自独立声明自己的绕序。

5.3 Cube（立方体）

[p.16]

立方体有 6 个面，每个面 2 个三角形 = 36 个三角形索引。8 个顶点。

```
Vector3[] vertices = new Vector3[8] {
    new Vector3(0,0,0), new Vector3(1,0,0),
    new Vector3(1,1,0), new Vector3(0,1,0), // 前面4个
    new Vector3(0,0,1), new Vector3(1,0,1),
    new Vector3(1,1,1), new Vector3(0,1,1) // 后面4个
};

int[] triangles = { /* 36个索引 */ };
```

5.4 法线问题：8 个顶点 vs 24 个法线

[p.17]

关键问题：立方体有 6 个面，每个面 4 个顶点，每个顶点需要指向该面外侧的法线。所以总共需要 **24 条法线**。但我们只定义了 **8 个顶点位置**！

矛盾根源：同一位置的顶点在不同面上需要不同的法线（硬边！）。立方体的角同时属于 3 个面，需要 3 条指向不同方向的法线。

解决方案：需要 **24 个顶点**（而非 8 个）。每个面的 4 个顶点独立声明，即使它们空间位置相同。这样：

- 8 个几何位置，但 24 个顶点数据（每个面 4 个独立顶点）
- 每个顶点指向其面外侧的法线
- 硬边效果正确渲染

6 程序化地形生成

6.1 生成平面

[p.19-20]

地形是一个有起伏的平面，平面由许多 Quad 拼接而成。

生成 $n \times n$ 平面的关键数据：

- **vertices:** $(n+1) \times (n+1)$ 个顶点（网格点）
- **triangles:** 每个小方格 2 个三角形，共 $n \times n \times 6$ 个索引
- **normals:** 初始全部朝上（`Vector3.up`）
- **uv:** 均匀分布覆盖 $[0,1]$ 范围

6.2 柏林噪声（Perlin Noise）

[p.21]

为了让顶点偏移看起来“随机但自然”，使用柏林噪声。

噪声：一系列 0-1 之间的（伪随机）值。

Perlin Noise 特性：

- 相邻采样点的值相近（平滑过渡），产生自然地形
- 可叠加不同频率（Octaves）增加细节
- Unity 提供 `Mathf.PerlinNoise(x, y)` 函数直接采样

地形偏移：以每个顶点的 UV 坐标采样 2D Perlin Noise（即 `Mathf.PerlinNoise(uv.x, uv.y)`），将噪声值作为垂直偏移量。

6.3 重新计算法线

[p.22]

顶点偏移后法线仍指向正上方，导致没有亮暗面之分（白花花一片）。使用 `mesh.RecalculateNormals()` 自动重新生成贴合地形的法线。

RecalculateNormals 原理：对每个顶点，获取周围几个点的位置做差。经过几次向量叉乘与求平均得到一个近似的法线值。

6.4 草的生成

[p.23]

遍历地形网格点，在每个点处生成几个小三角形作为草的模型，随机微调旋转/缩放以产生自然感。草的密度和随机性可以让草地看起来更真实。

进阶功能：风吹动画（通过 Shader 中的顶点动画）、草与角色互动（碰撞检测 + 位移）、更高效的 GPU Instancing 或几何着色器渲染。

7 课后习题详解

[p.24]

7.1 题 1：程序化生成正四棱锥

正四棱锥：底面正方形 + 4 个等腰三角形侧面（共 5 个面），5 个顶点（底面 4 个 + 顶点 1 个）。

```
Mesh CreatePyramid(float baseSize, float height) {
    Mesh mesh = new Mesh();
    Vector3[] vertices = new Vector3[5] {
        new Vector3(-baseSize/2, 0, -baseSize/2), // 底面后左
        new Vector3( baseSize/2, 0, -baseSize/2), // 底面后右
        new Vector3( baseSize/2, 0,  baseSize/2), // 底面前右
        new Vector3(-baseSize/2, 0,  baseSize/2), // 底面前左
        new Vector3(0, height, 0)                // 顶点
    };
    int[] triangles = new int[] {
        0,2,1, 0,3,2,    // 底面（两个三角形）
        0,1,4,           // 后面
        1,2,4,           // 右面
        2,3,4,           // 前面
        3,0,4            // 左面
    };
    mesh.vertices = vertices;
    mesh.triangles = triangles;
    mesh.RecalculateNormals();
    return mesh;
}
```

7.2 题 2：使用柏林噪声生成动画海面

```
void Update() {
    Vector3[] vertices = mesh.vertices;
    for (int i = 0; i < vertices.Length; i++) {
        float x = vertices[i].x * noiseScale;
```

```
        float y = vertices[i].z * noiseScale + Time.time * speed;
        vertices[i].y = Mathf.PerlinNoise(x, y) * waveHeight;
    }
    mesh.vertices = vertices;
    mesh.RecalculateNormals();
}
```

关键技巧：在噪声采样中加入 `Time.time` 作为第三维的采样偏移，使噪声值随时间变化产生波浪动画效果。

7.3 题 3：在任意位置生成草

需要在三角形面上使用**重心坐标**（**Barycentric Coordinates**）在任意位置插值出草的位置：

```
Vector3 GetRandomPointOnTriangle(Vector3 v0, Vector3 v1, Vector3 v2) {
    float r1 = Random.value;
    float r2 = Random.value;
    if (r1 + r2 > 1) { r1 = 1 - r1; r2 = 1 - r2; }
    return v0 + r1*(v1-v0) + r2*(v2-v0);
}
```

对地形中每个三角形面随机采样若干点种草，而非仅限于顶点。